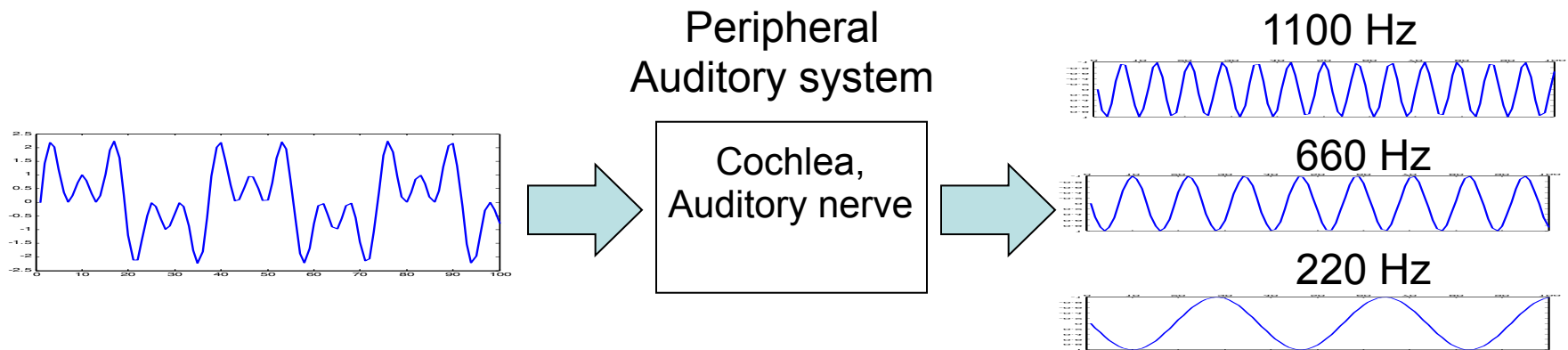


Topic 6

The Digital Fourier Transform

Why bother?

- The ear processes sound by decomposing it into sine waves at different frequencies.
- An algorithm that does the same would be a step towards machines that hears things as humans do.
- So...how do we do this by machine?



Jean Baptiste Joseph Fourier

He was a French mathematician and physicist who lived from 1768-1830.

He presented a paper in 1807 to the Institut de France claiming any continuous signal with finite period could be represented as the sum of an infinite series of properly chosen sinusoidal wave functions at different frequencies.

We now call this the Fourier Series.

Could this be the way to decompose sounds into different frequencies, like the ear does?

Factoid

Among the reviewers of Fourier's paper were two famous mathematicians

- Joseph Louis Lagrange (1736-1813)
- Pierre Simon de Laplace (1749-1827)

Lagrange said sine waves could not perfectly represent signals with discontinuous slopes, like square waves. (He was, technically, right)

Thus, the Institut de France did not publish Fourier's work until 15 years later, after Lagrange died.

The Fourier Series

Given a periodic function $x(t)$

$$x(t + mT) = x(t) \quad \forall m \in \text{Integers}$$

$\uparrow$
the period

An **INFINITE** series of sine and cosine functions reproduces $x(t)$

$$x(t) = A_0 + \sum_{n=1}^{\infty} [A_n \cos(\omega_n t) + B_n \sin(\omega_n t)]$$

Where frequency ω_n is an integer multiple n of fundamental frequency ω_0

$$\omega_n = n\omega_0 = 2\pi n / T$$

Aside: Two defs for “frequency”

- The frequency of a periodic function can be defined in two ways.

$$f = \omega / 2\pi = 1 / T$$

The diagram illustrates the relationship between three quantities in the equation $f = \omega / 2\pi = 1 / T$. Below the equation, three labels are positioned: "Frequency" under f , "Angular Frequency" under ω , and "Period" under T . Each label has a vertical arrow pointing upwards to its corresponding variable in the equation.

Fourier Series

The diagram illustrates the Fourier Series equation with several annotations and arrows:

- The original signal**: An arrow points from this text to $x(t)$ on the left side of the equation.
- Quite a few!**: An arrow points from this text to the summation symbol $\sum_{n=1}^{\infty}$, indicating the infinite number of components.
- Amplitude: the contribution of this component**: Two arrows point from this text to A_n and B_n , which represent the amplitudes of the cosine and sine components, respectively.
- The frequency of this component**: Two arrows point from this text to ω_n in both $\cos(\omega_n t)$ and $\sin(\omega_n t)$, indicating the angular frequency of each component.

$$x(t) = A_0 + \sum_{n=1}^{\infty} \left[A_n \cos(\omega_n t) + B_n \sin(\omega_n t) \right]$$

PROBLEM 1

The Fourier Series has an infinite number of sinusoids in it.

It isn't practical to calculate an infinite number of things, in the general case.

We need to frame the problem as a finite one, so we can actually solve the general case.

PROBLEM 2

A function representable by a Fourier series is perfectly periodic. Therefore, it goes on infinitely.

Real world audio is not infinite in length...and things are only locally periodic.

Solution: Sample, Window, Hope

STEP 1: Take a “snapshot” of the signal by sampling it a finite number of times within a brief time window

STEP 2: Pretend what you saw in that window goes on forever

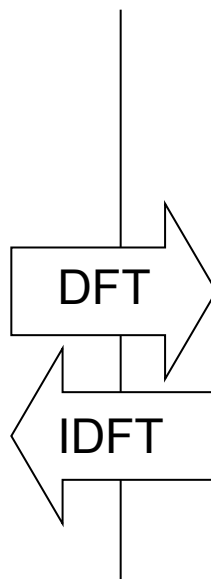
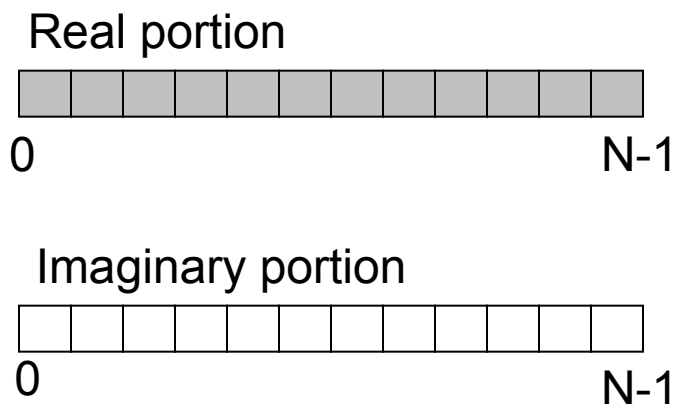
Voila! Now you have a “periodic” signal represented by a finite number of points.

Discrete Fourier Transform

- Represents a **finite** sequence of complex values as a **finite** number of discrete sinusoidal functions.
- This finite sequence of samples can be perfectly reproduced by the finite set of sinusoids.

Time domain

$x[n]$



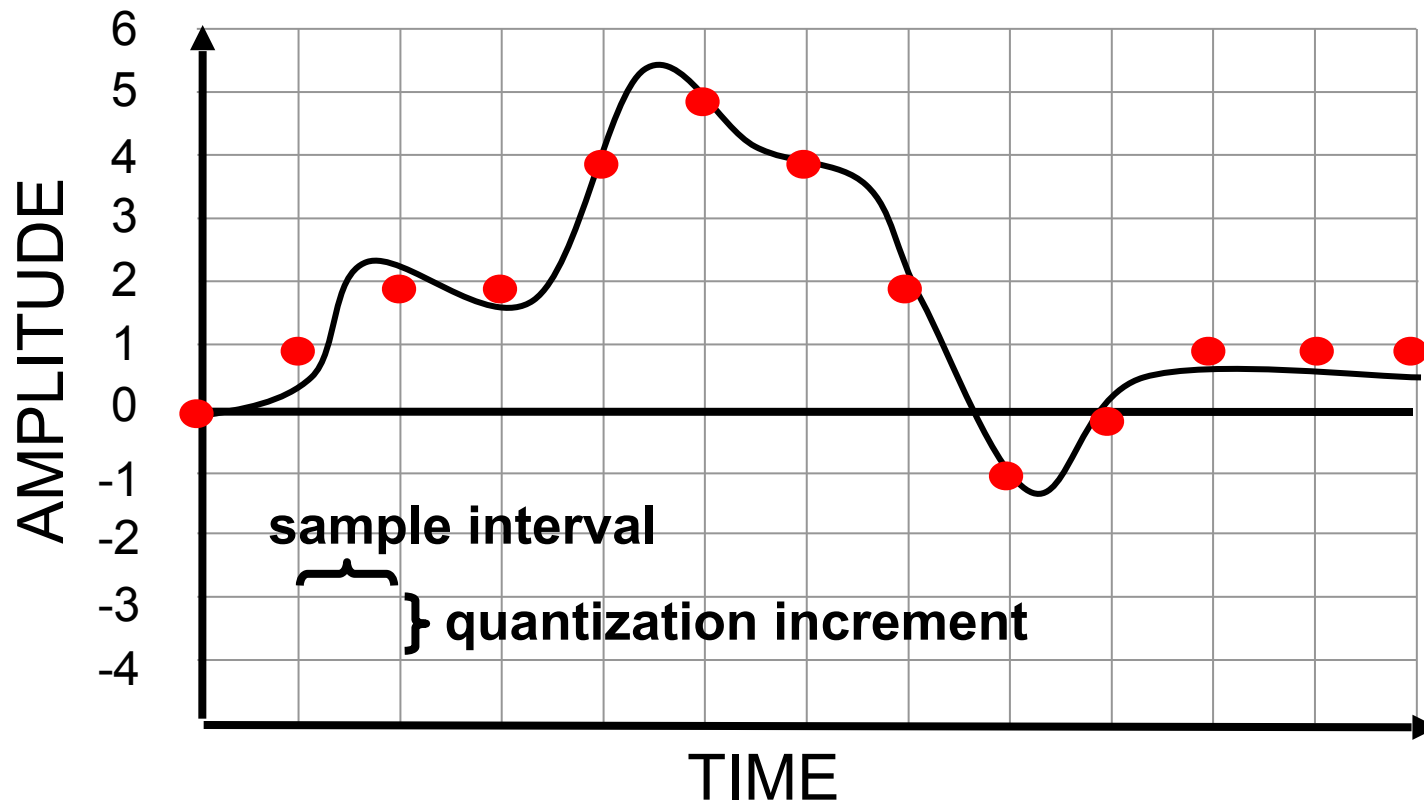
Frequency domain

$X[k]$



Digital Sampling

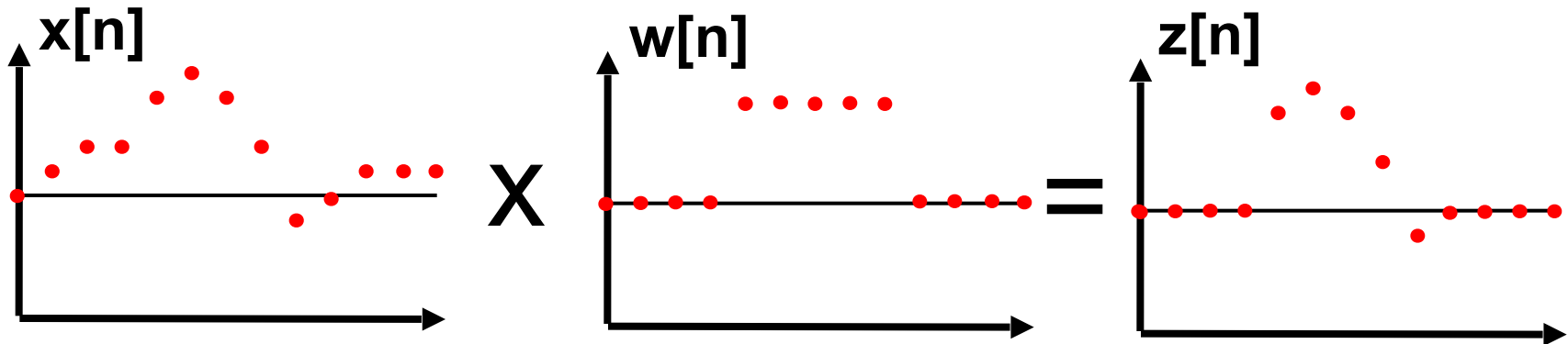
An analog signal is ***sampled*** into sequence of discrete sample points, $x[n]$



Windowing

$\mathbf{x[n]}$ is *windowed* by function $\mathbf{w[n]}$

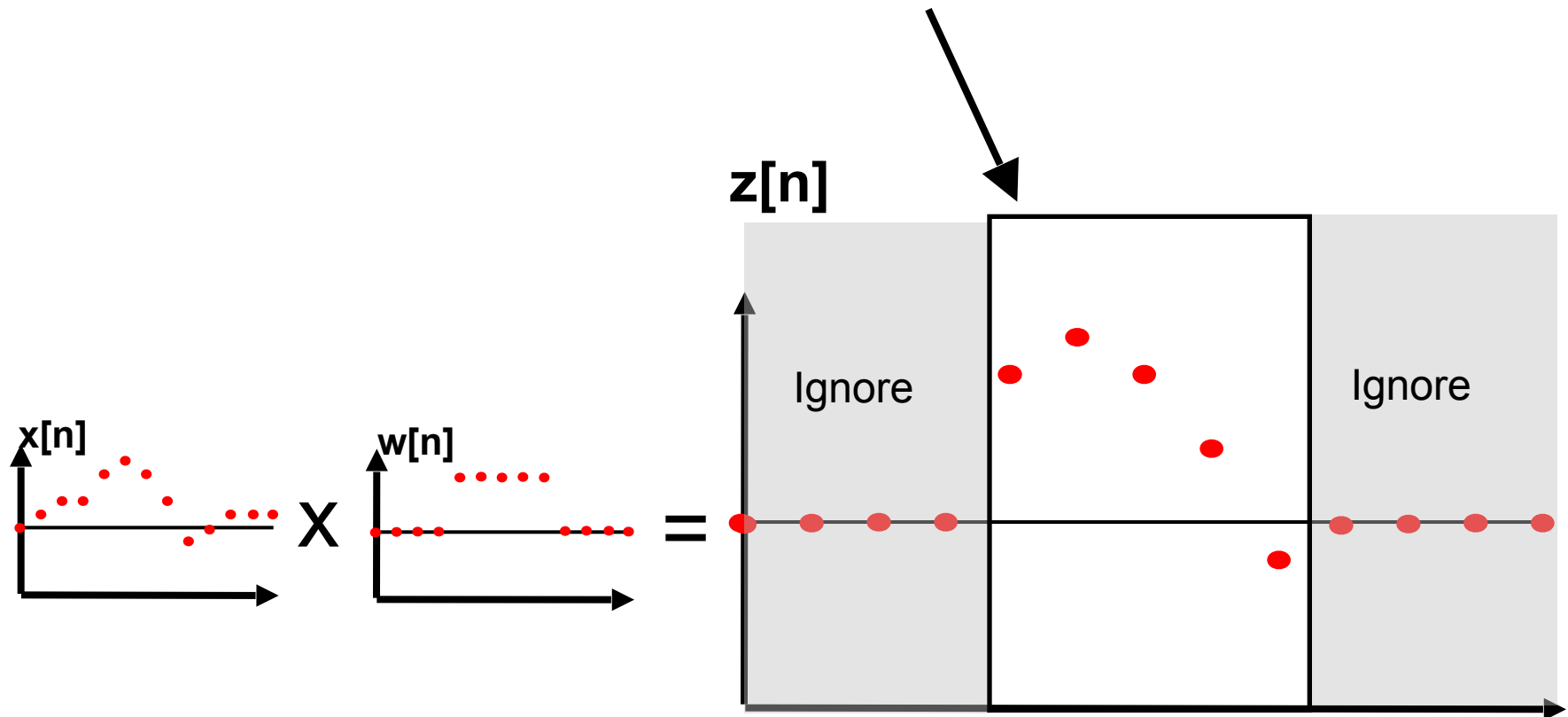
(multiply the *i*th value of $\mathbf{x}$ by the *i*th value of $\mathbf{w}$)



Example: windowing $x[n]$ with a rectangular window

Only what's in the window

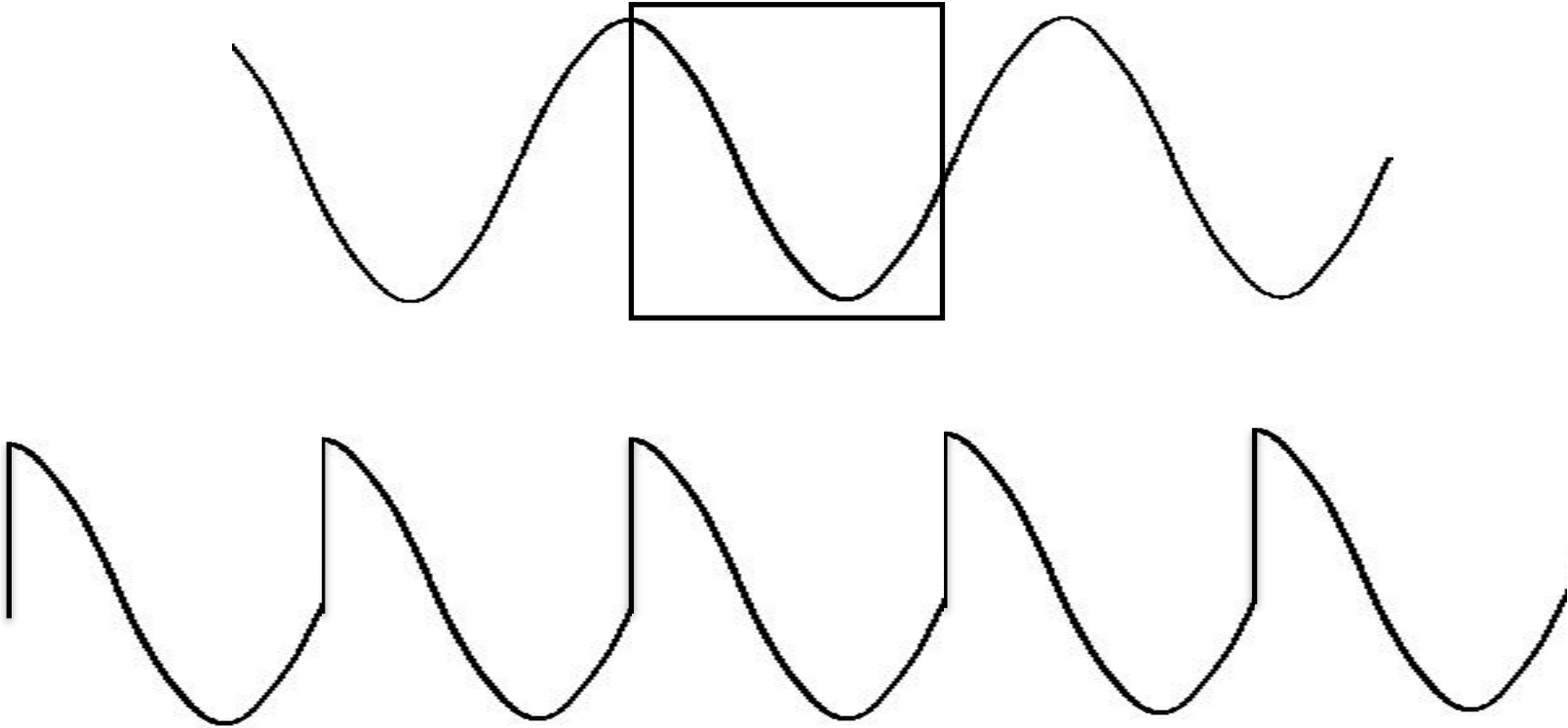
Do the DFT only on the values in the window



Windowing can lead to problems

The original signal

The sample window



What the Fourier Transform “imagines” the signal looks like,
based on what was in the window

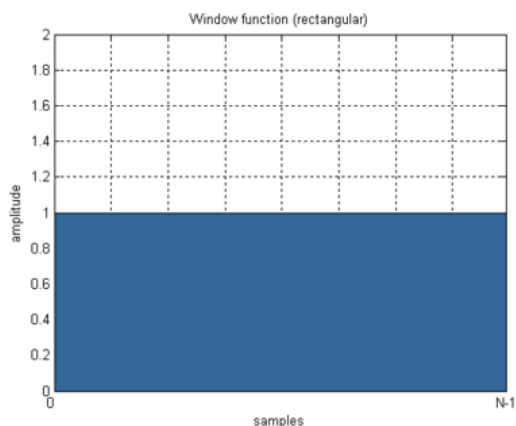
Window size

- Your window should be longer than the period of the function you want to analyze
- At a sample rate of 8000 Hz, what is the minimum window size that can capture the lowest sound you can hear?

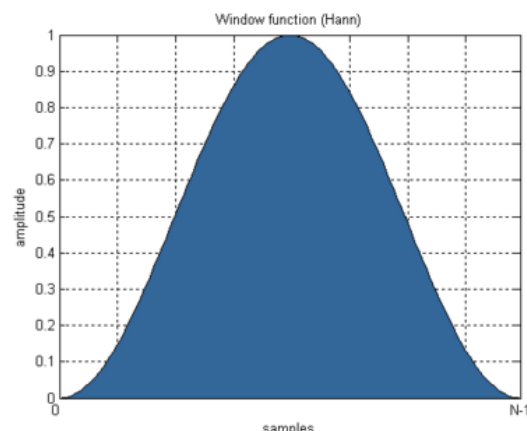
Window shape

- Making the window “small” at the edges reduces weirdness.

BAD FOR GETTING
A GOOD IDEA OF
WHAT'S GOING ON



BETTER FOR GETT
ING A GOOD IDEA



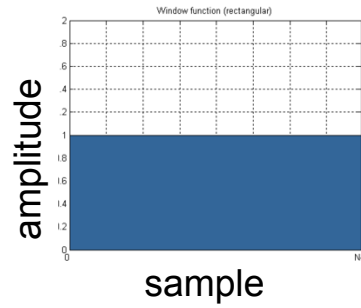
Why window shape matters

- Don't forget that a DFT assumes the signal in the window is periodic
- The boundary conditions mess things up... unless you manage to have a window whose length is an exact integer multiple of the period of your signal
- Making the edges of the window less prominent helps suppress undesirable artifacts

Some famous windows

- Rectangular

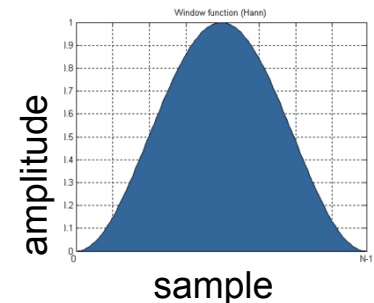
$$w[n] = 1$$



Note: we assume $w[n] = 0$ outside some range $[0, N]$

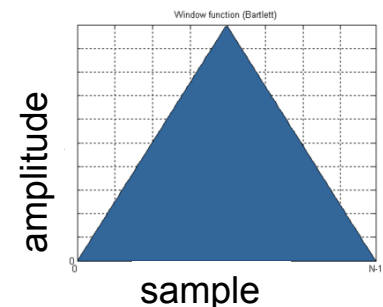
- Hann (Julius von Hann)

$$w[n] = 0.5 \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right)$$



- Bartlett

$$w[n] = \frac{2}{N-1} \left(\frac{N-1}{2} - \left| n - \frac{N-1}{2} \right| \right)$$



The DFT and IDFT formulae

What's different
between them

Discrete
Fourier
Transform

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-\frac{2\pi i}{N} kn}$$

Inverse
Discrete
Fourier
Transform

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{\frac{2\pi i}{N} kn}$$

Some points

- If you have n samples in your window, you will get n frequencies of analysis from your FFT.
- If the samples were grabbed at regular times, then the analysis frequencies will be spaced regularly in the frequency domain.

Fundamental Freq of a Signal

- This is NOT the DC offset.
- This is NOT fundamental frequency of analysis of an FFT
- The lowest frequency a sine or cosine can have and still fit into one period of the signal function.
- Often called “F zero” and written F0

$$f_0 = \omega_0 / 2\pi = 1 / T$$

Fundamental Frequency of Analysis

- The lowest frequency component a Fourier transform can analyze meaningfully
- All the frequencies of analysis are integer multiples of the fundamental frequency of analysis
- This is also the spacing between frequencies of analysis.

$$f_{analysis} = \frac{S}{N}$$

← Sample rate

← Number of samples in my window

23

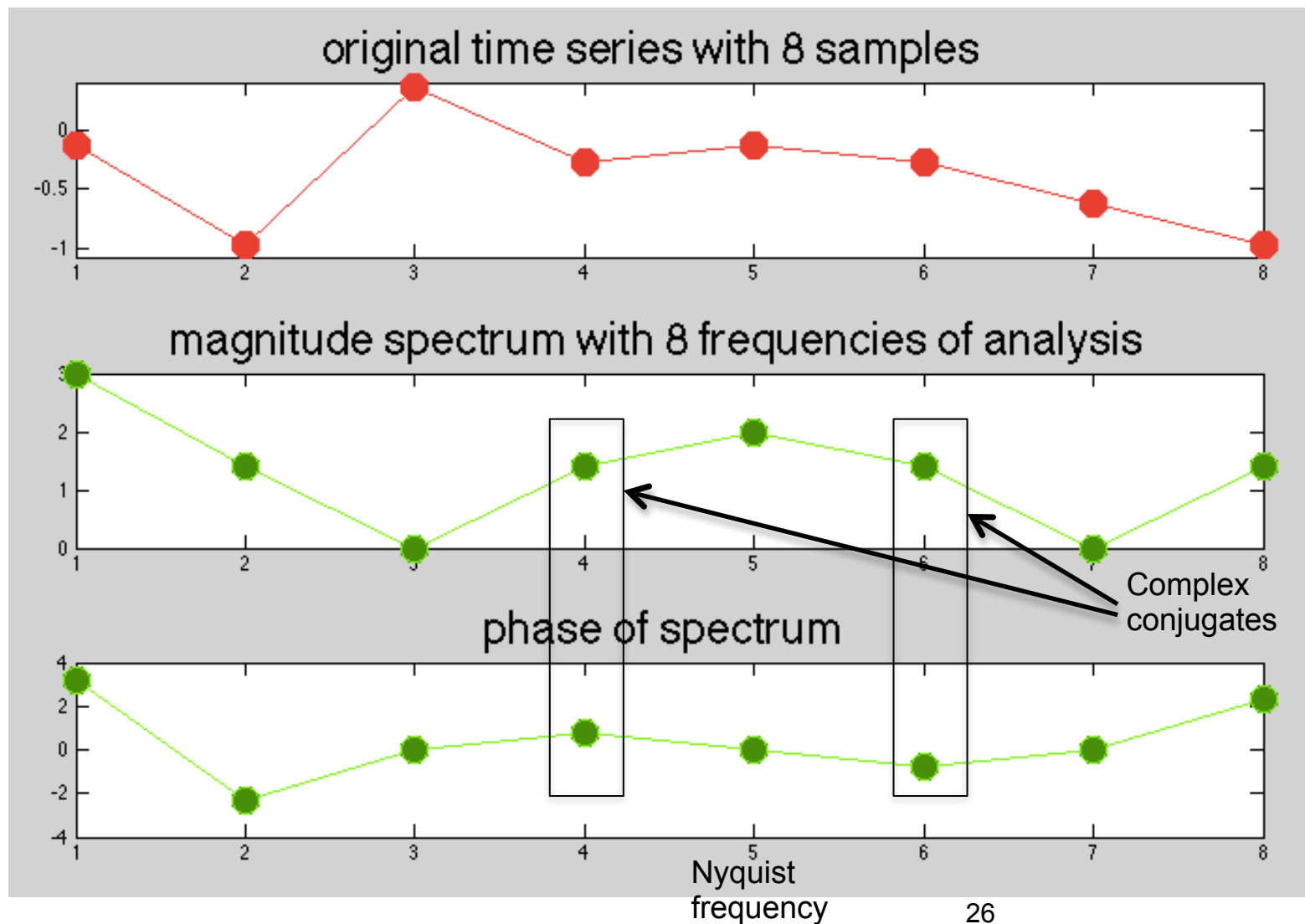
About Frequencies of Analysis

- A recording with **N** points produces a DFT with **N** points.
- The energy in the DFT is symmetric around two “pivot points”: the DC offset (the 0 frequency) and $\frac{1}{2}$ the sample frequency (**S**).
- Let’s look at an example.

The numbers in an 8 point FFT

Array Index	0	1	2	3	4	5	6	7
8 point signal	-0.13	-0.98	0.38	-0.27	-0.13	-0.27	-0.63	-0.98
FFT of signal	-3 -i	-1-i	0	1+i	2	1-i	0	-1+i
DC offset Nyquist frequency Complex conjugates								
Better index	0	1	2	3	4 (-4)	-3	-2	-1
Frequency of FFT	0	S/N	2S/N	3S/N	4S/N	-3S/N	-2S/N	-S/N
If S=1000 Hz and N = 8	0	125Hz	250Hz	375Hz	500Hz	-375Hz	-250Hz	-125Hz

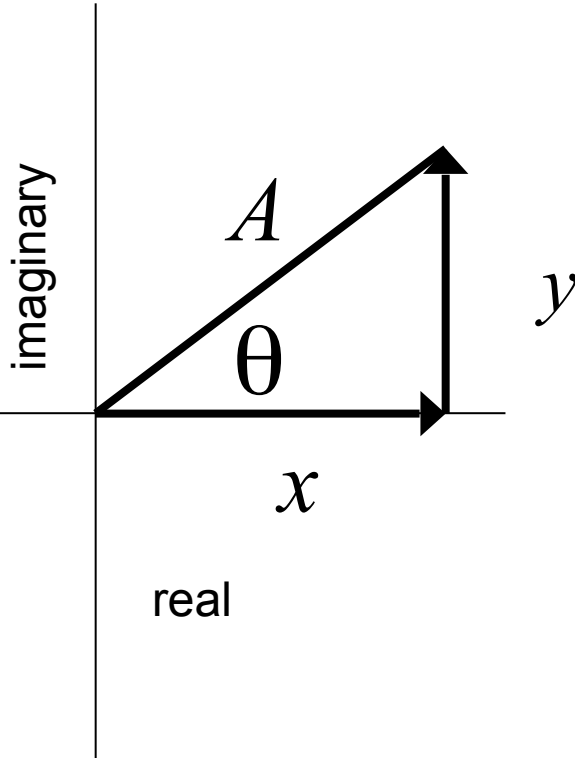
Another view of the same data



Complex Numbers

$$z \equiv x + iy$$

$$\equiv A(\cos\theta + i\sin\theta)$$



$$x = A \cos\theta$$

$$y = A \sin\theta$$

$$A = \sqrt{(x^2 + y^2)}$$

Euler's Formula

- Useful for relating polar coordinates to rectangular coordinates

$$e^{i\theta} = \cos\theta + i\sin\theta$$

Thus...

$$z = Ae^{i\theta}$$

AMPLITUDE

PHASE

Multiplying Complex Numbers

- POLAR notation EASIER for this

$$z_1 = A_1 e^{i\theta_1} \quad z_2 = A_2 e^{i\theta_2}$$

$$z_3 = (A_1 A_2) e^{i(\theta_1 + \theta_2)}$$

Multiplying Complex Numbers

- Cartesian works as follows...

$$z_1 = x_1 + iy_1 \quad z_2 = x_2 + iy_2$$

$$z_3 = x_1x_2 - y_1y_2 + i(x_1y_2 + x_2y_1)$$

Complex Conjugate

- the **complex conjugate** of a complex number is given by changing the sign of the imaginary part.

A complex number

$$z = a + ib$$

Its complex conjugate

$$\overline{z} = a - ib$$

DFT in Cartesian Coordinates

$$\begin{aligned} X[k] &= \sum_{n=0}^{N-1} x[n] e^{-\frac{2\pi i}{N} kn} \\ &= \sum_{n=0}^{N-1} x[n] \left(\cos\left(-\frac{2\pi kn}{N}\right) + i \sin\left(-\frac{2\pi kn}{N}\right) \right) \end{aligned}$$

I got this using Euler's formula.

In Cartesian coordinates, the fact that these are complex values is more obvious.

The Inverse DFT

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{\frac{2\pi i}{N} kn}$$

**complex
number**

$$= \frac{1}{N} \sum_{k=0}^{N-1} X[k] \left(\cos \left(\frac{2\pi kn}{N} \right) + i \sin \left(\frac{2\pi kn}{N} \right) \right)$$

Computational complexity

- How many operations does this take for each frequency?
- How many operations total?

$$X[k] = \sum_{n=0}^{N-1} x[n] \left(\cos \left(-\frac{2\pi kn}{N} \right) + i \sin \left(-\frac{2\pi kn}{N} \right) \right)$$

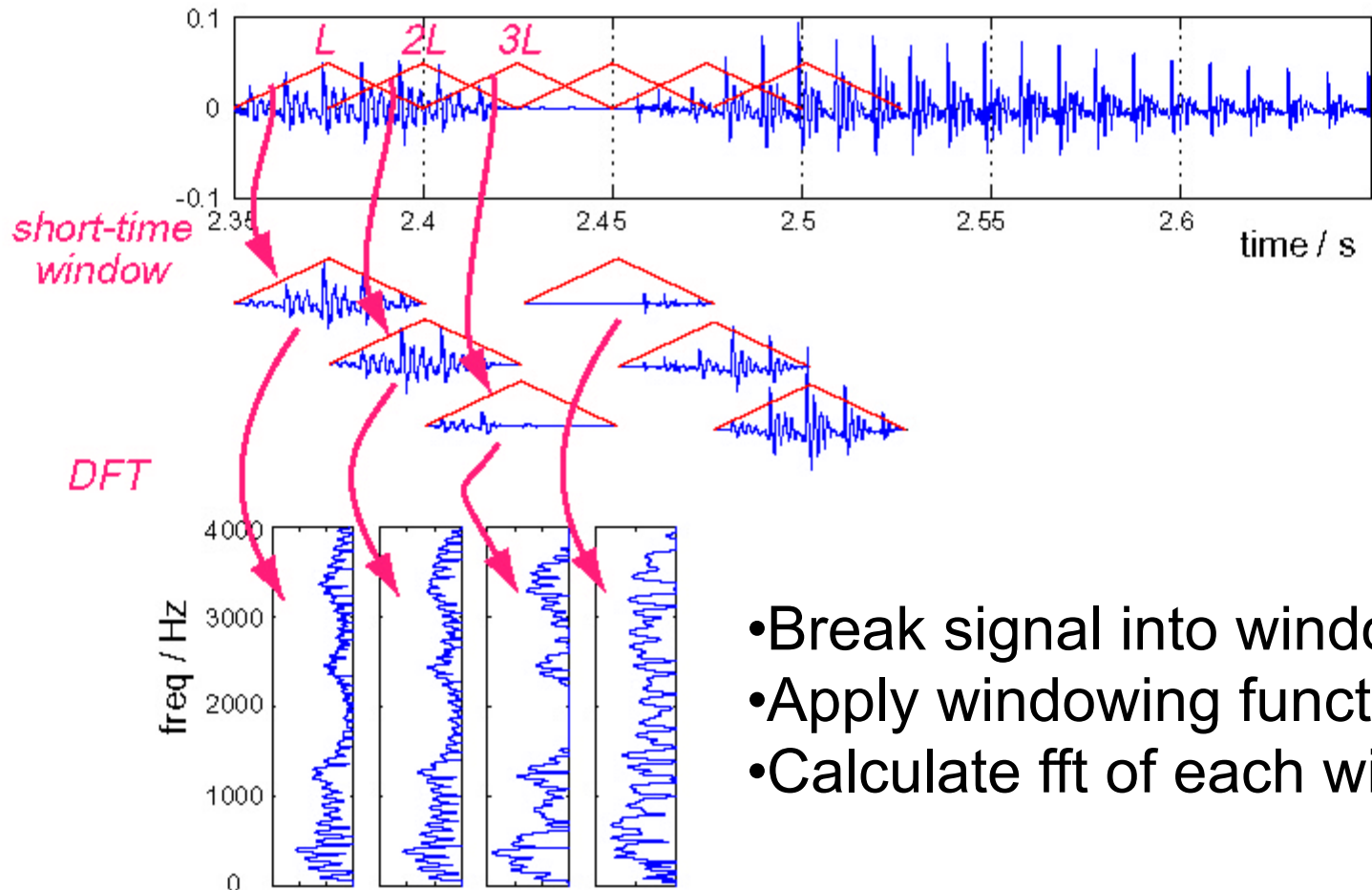
The FFT

- Fast Fourier Transform
 - A much, much faster way to do the DFT
 - Introduced by Carl F. Gauss in 1805 (ish)
 - Rediscovered by Cooley & Tukey in 1965
 - Big O notation for this is **$O(N \log N)$**
 - Matlab functions **fft** and **ifft** are standard
 - **REQUIRES** the window size be a power of 2

Steps when making a spectrogram

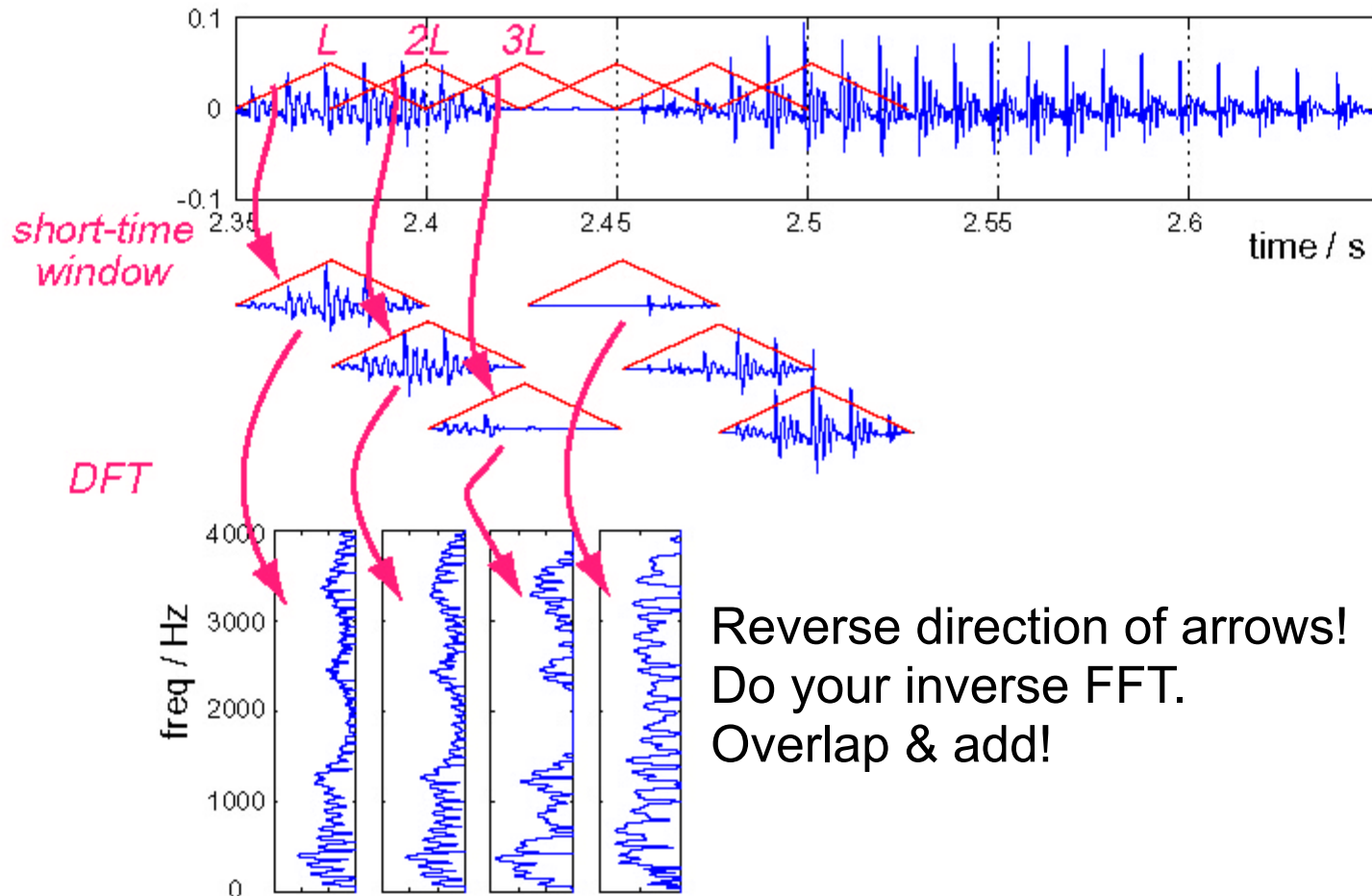
- Cut up a signal into (possibly overlapping) windows
- Apply a windowing function (see slide 14), if you want to remove those edge effects
- Take the FFT of each window
- if X is the FFT, do `np.abs(X)` to get the magnitude of the signal
- For display, take the log of the values (decibels would be a good way)
- Only display frequencies up to $1/2$ the sample rate
- Plot it, where vertical = freq (low frequency = lower on the graph), horizontal = time step (left is earlier, right is later) and color = how loud

Short time Fourier Transform



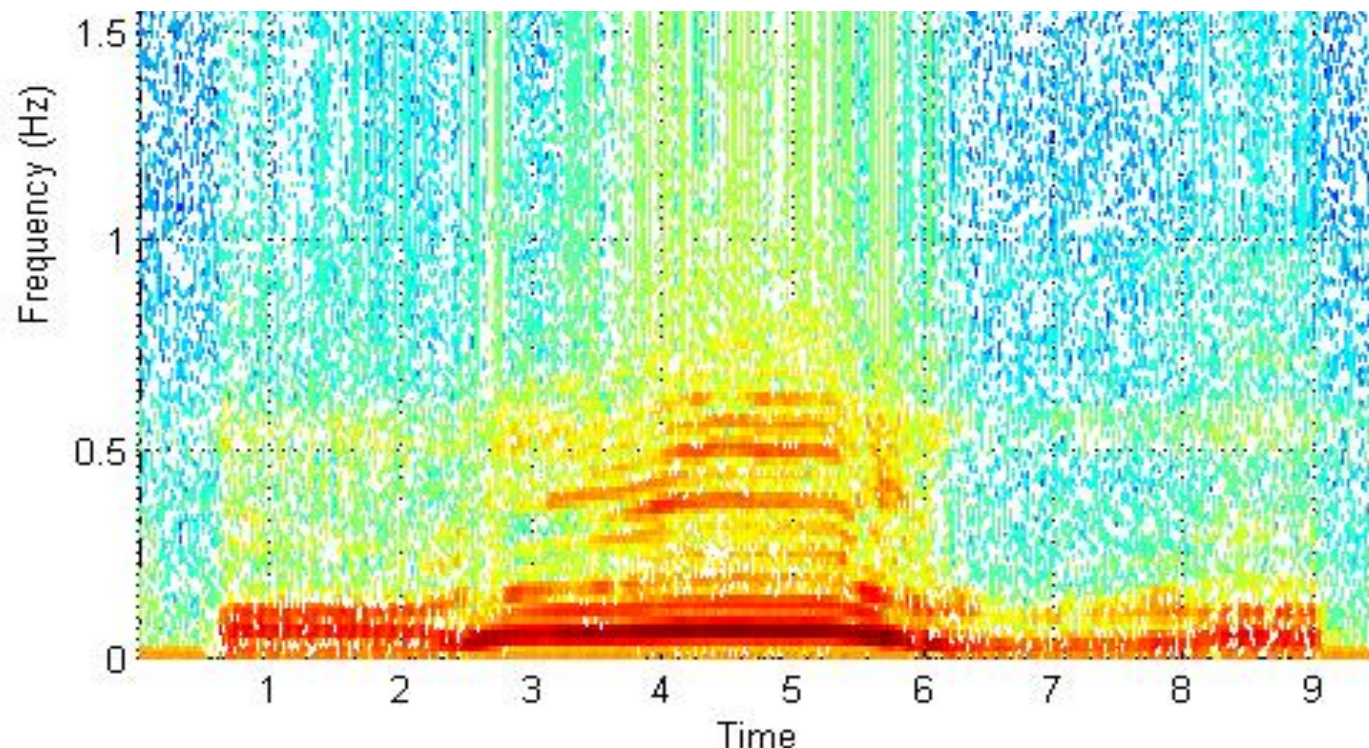
- Break signal into windows
- Apply windowing function
- Calculate fft of each window

Inverse Short time Fourier Transform



The Spectrogram

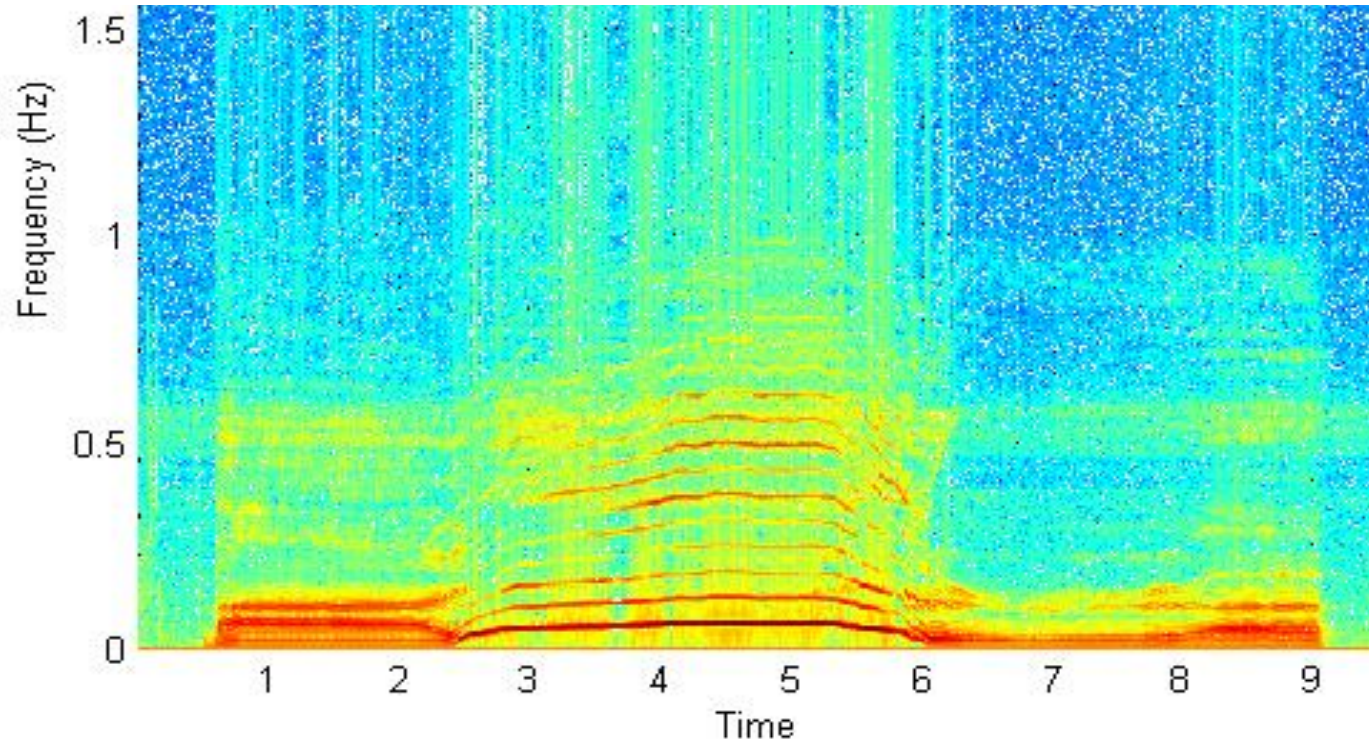
`spectrogram(y,256,128,256,fs,'yaxis');`



- A series of short term DFTs
- Typically just displays the magnitudes in X of the frequencies up to $\frac{1}{2}$ sample rate
- There is a **spectrogram** function³⁹ in matlab

The Spectrogram

`spectrogram(y,1024,512,1024,fs,'yaxis');`



- A series of short term DFTs
- Typically just displays the magnitudes in X of the frequencies up to $\frac{1}{2}$ sample rate
- There is a **spectrogram** function⁴⁰ in matlab